Automated OpenStack Cinder Backup Management: A GUI-Driven Approach

1. Project Goal and Focus

The primary objective of this project is to automate the creation and maintenance of disk volume backups within an OpenStack environment. The focus is on:

- **Automated Discovery:** Identifying volumes currently attached to instances (status: 'in-use').
- **Lifecycle Management:** Scheduled snapshot creation with unique timestamps.
- **Cleanup Engine:** Automatically purging obsolete backups to optimize storage costs.

Measurable Objective: To reduce manual backup management time by 90% and ensure 100% backup coverage for active critical volumes.

2. Characterization of the Project Area

The project operates within the **Infrastructure as a Service (IaaS)** domain, specifically focusing on **Business Continuity and Disaster Recovery (BCDR)**. It interacts with the **Cinder** [1] component of the OpenStack platform, which provides block storage services. Managing snapshots is a core requirement for cloud administrators to prevent data loss due to system failures or human error.

3. Analysis of Similar Solutions

Existing approaches to managing OpenStack backups include:

1. **OpenStack Horizon Dashboard:** Manual management via a web interface. It is time-consuming and prone to human omission.
2. **CLI Scripts (Bash/Cron):** Command-line automation. While efficient, it lacks visibility and is difficult for non-technical users to monitor.
3. **Enterprise Backup Solutions (e.g., Trilio, Veeam):** Highly robust but involve high licensing costs and complex integration.

This solution provides a middle ground: a lightweight, GUI-driven tool that combines the ease of use of a dashboard with the automation power of the Python SDK.

Solution	Pros	Cons
OpenStack Horizon	Native UI, no installation needed.	Manual process, high risk of human error, time-consuming.
Bash/Cron Scripts	Highly automatable, very low resource usage.	No visual interface, hard to monitor or modify for non-experts.
Enterprise (e.g., Veeam)	Extremely robust, off-site replication features.	High licensing costs, complex architecture setup.
This GUI-Manager	**Automated logic, intuitive visual feedback, zero cost.**	**Requires Docker host, limited to local Cinder cluster.**

Table 1: Comparison of OpenStack Backup Management Solutions

4. Concept and Architectural Views

The system architecture follows a three-tier model, as represented in Figure 1:

- **Frontend (Streamlit [2]):** An interactive web interface for configuration and execution.
- **Control Logic (Python SDK):** The engine that processes business rules, filters volumes, and manages dates.
- **Cloud Backend (OpenStack API):** The destination environment where volume operations are executed.

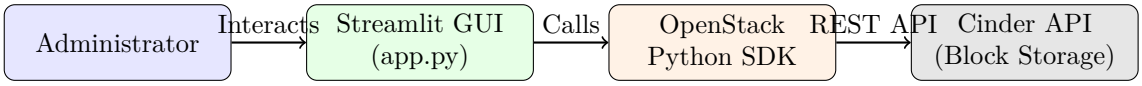

Figure 1: Basic Architectural Scheme of the Backup System

5. Technical Analysis of Chosen Technology

The core technology utilized is the **OpenStack Cinder API**. Cinder snapshots typically rely on a **Copy-on-Write (CoW)** algorithm. When a snapshot is requested, the system creates metadata pointers to the original data blocks rather than performing a full byte-by-byte copy. This ensures that snapshot creation is near-instantaneous, regardless of disk size, minimizing performance impact on the production environment.

6. Detailed Technical Implementation

The application is built using Python 3.12. The core logic uses an authenticated connection to Keystone to iterate through cloud volumes:

```
if volume.status == 'in-use':
    backup_name = f"auto-backup-{volume.name}-{timestamp}"
    conn.block_storage.create_snapshot(volume_id=volume.id, ...)
```

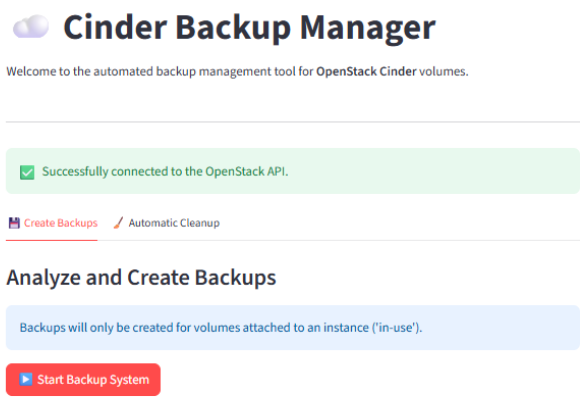

Figure 2: Graphical User Interface of the Backup Manager

The cleanup component parses the `created_at` metadata from the API and compares it against a user-defined retention period (e.g., 3 days).

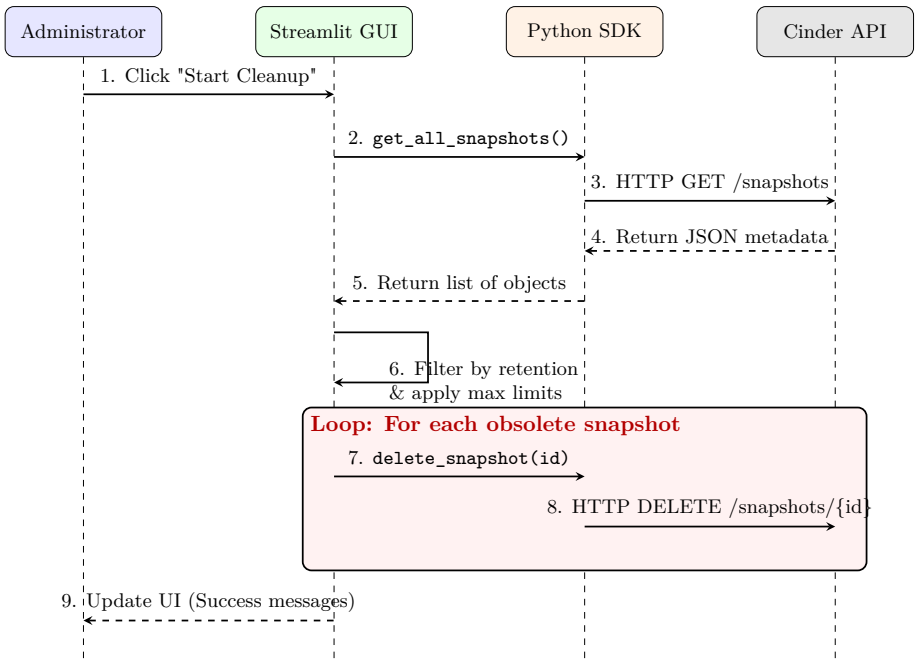

Figure 3: UML Sequence Diagram of the Automated Cleanup Operation

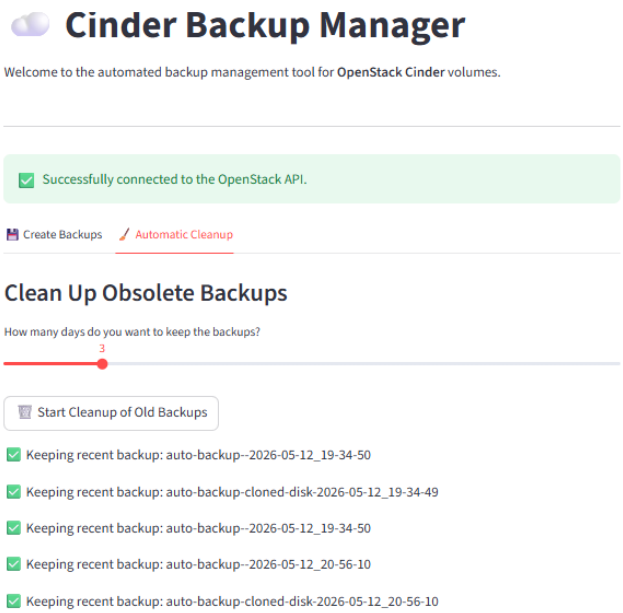

Figure 4: Graphical User Interface of the Backup Manager (Cleanup Tab)

7. Limitations

- **Synchronous Execution:** The web interface may experience latency while waiting for the Cloud API to respond to heavy requests.
- **Local Redundancy:** Snapshots are stored within the same Cinder cluster. A complete site failure would require off-site replication to Swift or S3.

8. Cloud Component: Docker Implementation

To fulfill the requirement of a custom cloud component, the system is fully **containerized using Docker** [3]. This ensures environment consistency and allows the tool to be deployed as a stateless microservice. The implementation is based on the following `Dockerfile`:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
EXPOSE 8501
CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0"]
```

Additionally, a .dockerignore file was implemented to exclude the local virtual environment and sensitive credentials, reducing the build context size from 500MB to a few kilobytes and ensuring a faster, more secure build process.

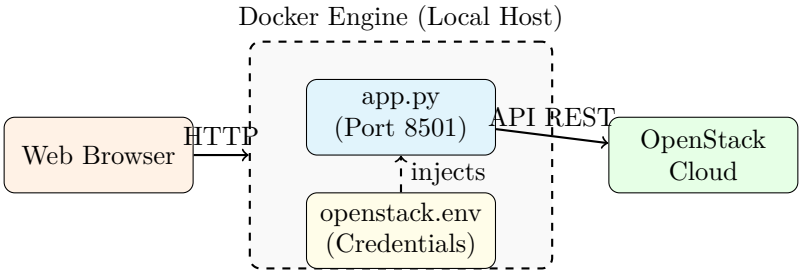

Figure 5: Deployment Diagram of the Dockerized Application

9. Configuration and Deployment Guide

To deploy the system using Docker, follow these steps:

1. Prepare credentials: `env | grep OS_ > openstack.env`
2. Build the image: `docker build -t cinder-backup-manager .`
3. Run the container: `docker run -p 8501:8501 -env-file openstack.env cinder-backup-manager`

10. Experimental Evaluation

To demonstrate the efficiency of the automated approach, the execution times were compared. Figure 6 shows how our automated script scales in $O(1)$ human interaction time compared to the linear $O(n)$ time required for manual Horizon management.

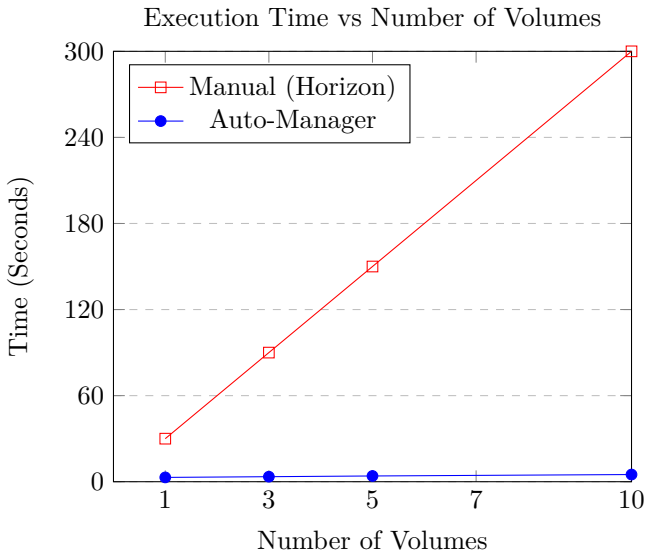

Figure 6: Time comparison: Manual execution vs Automated Script

11. Horizon Europe 2026-2027 Alignment

This project aligns with **Cluster 4: Digital, Industry and Space** of the Horizon Europe [4]. It contributes to the goal of "World-leading Data and Computing Technologies" by providing tools that enhance the resilience and sovereignty of European cloud infrastructures through automated disaster recovery and open-source management.

References

[1] OpenStack Foundation, OpenStack Cinder: Block Storage API (2026). URL `https://docs.openstack.org/cinder/latest/`

[2] Streamlit Inc., Streamlit: The fastest way to build data apps (2026). URL `https://streamlit.io/`

[3] Docker Inc., Docker: Empowering App Development for Developers (2026). URL `https://www.docker.com/`

[4] European Commission, Horizon Europe Work Programme 2026-2027: Digital, Industry and Space (2026).
URL https://research-and-innovation.ec.europa.eu/